

Platform Engineering & DevOps

¿Quién usa qué en Terraform?

Construyendo IaC
Observability

Port + GitHub + TFC

Diana Alfaro

Información adicional

DEVOPSDAYS.PE



27 - 28 AGO 2026

Perfil del speaker



Diana Alfaro

Cloud Solutions Architect AWS – Yape

💻 AWS Community Builder: DevTools

✨ Co-Líder de la comunidad AWS UG Caracas 🇨🇴

💕 Co-Organizadora de AWS Girls Perú 🇵🇪

👨🏫 Docente del programa AWS re/Start



CLOUD

DEVOPS

AWS

ARQUITECTURA



¿Cuál era el problema?

¿Puedo actualizar este módulo sin romper producción?

ecs-service v1.2.0 → ??? repos

Una pregunta que casi nadie puede responder con certeza al escalar Terraform en múltiples equipos

- 40+ repos Terraform · múltiples equipos · versiones distintas
- Sin visibilidad = conocimiento tribal
- Auditorías manuales: siempre desactualizadas



¿Qué es un IDP?

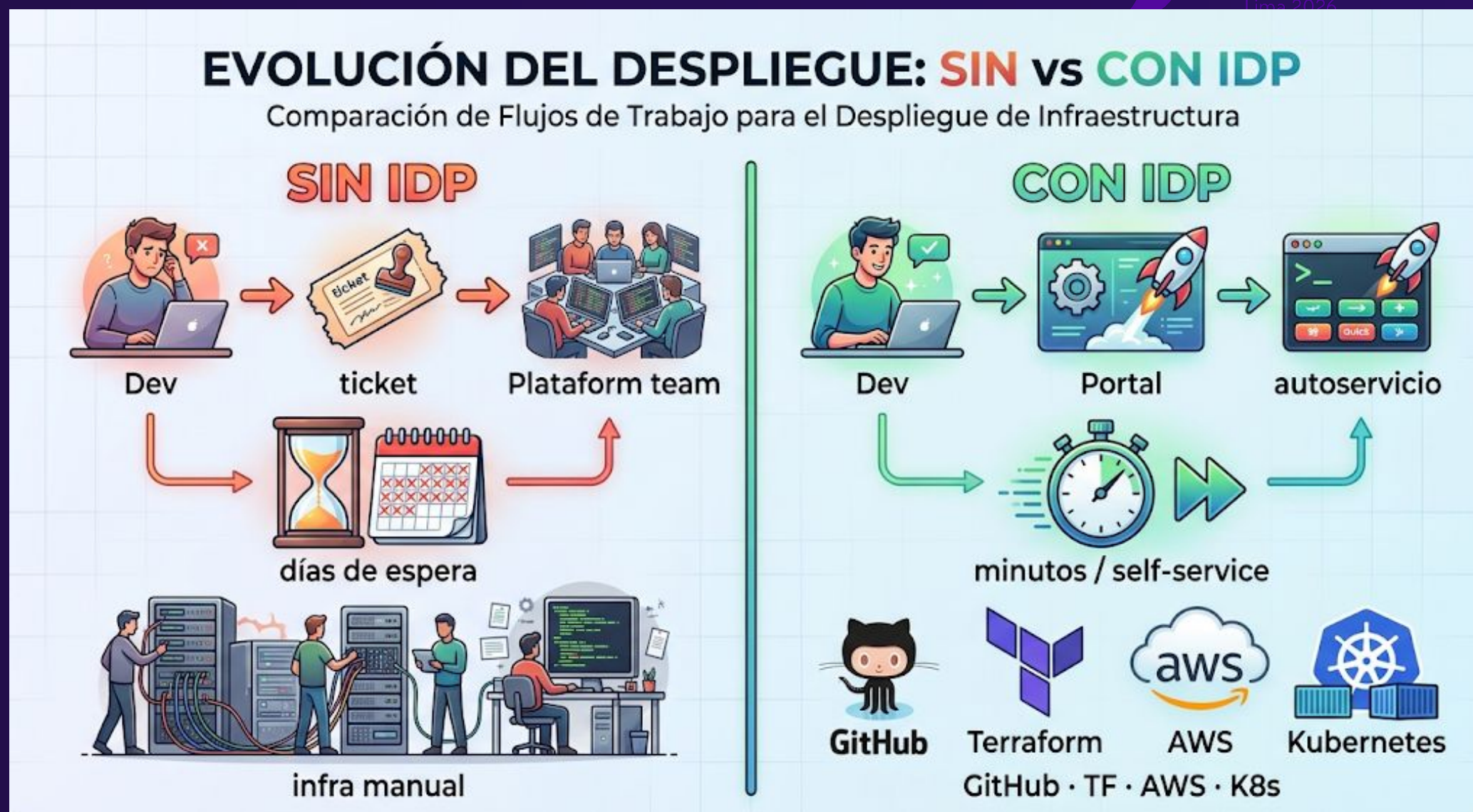
La capa de autoservicio que abstrae la complejidad de infraestructura para los equipos

No es un producto, es una práctica — y Port es la plataforma con la que lo implementamos

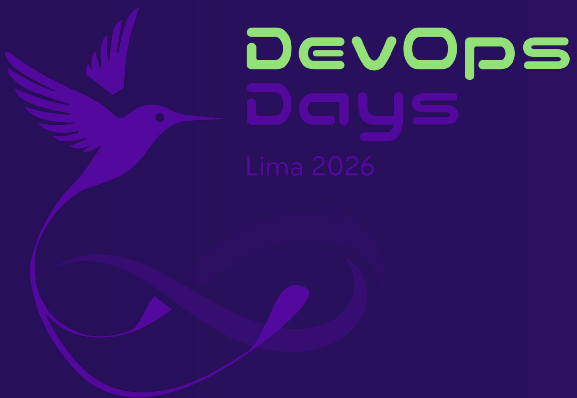
- Catálogo centralizado de servicios, repos e infraestructura
- Autoservicio: los devs hacen sin depender del equipo de plataforma
- Visibilidad: dashboards y governance sobre lo que corre en producción



DevOps
Days
Lima 2024



Port: el Developer Portal



Un catálogo de software modelado con blueprints, entidades y relaciones
Conecta datos de GitHub, Terraform Cloud, Jira y cualquier fuente — sin código de UI

BLUEPRINTS

{ schema }

Define el tipo de entidad

Como una tabla en una base de datos

Ejemplos:

terraformModule

repo_tf_module

githubRepository

squad

ENTITIES

[instance]

Las instancias reales de cada blueprint

Como una fila en esa tabla

Ejemplos:

vpc/v2.1.0 → módulo

repo-api → githubRepository

uso de vpc en repo-api

equipo-infraestructura

RELATIONS

A — B

El grafo que conecta blueprints entre sí

Quién usa qué, quién es responsable

Ejemplos:

¿Qué repos usan vpc v2.1.0?

¿Qué versión usa repo-api?

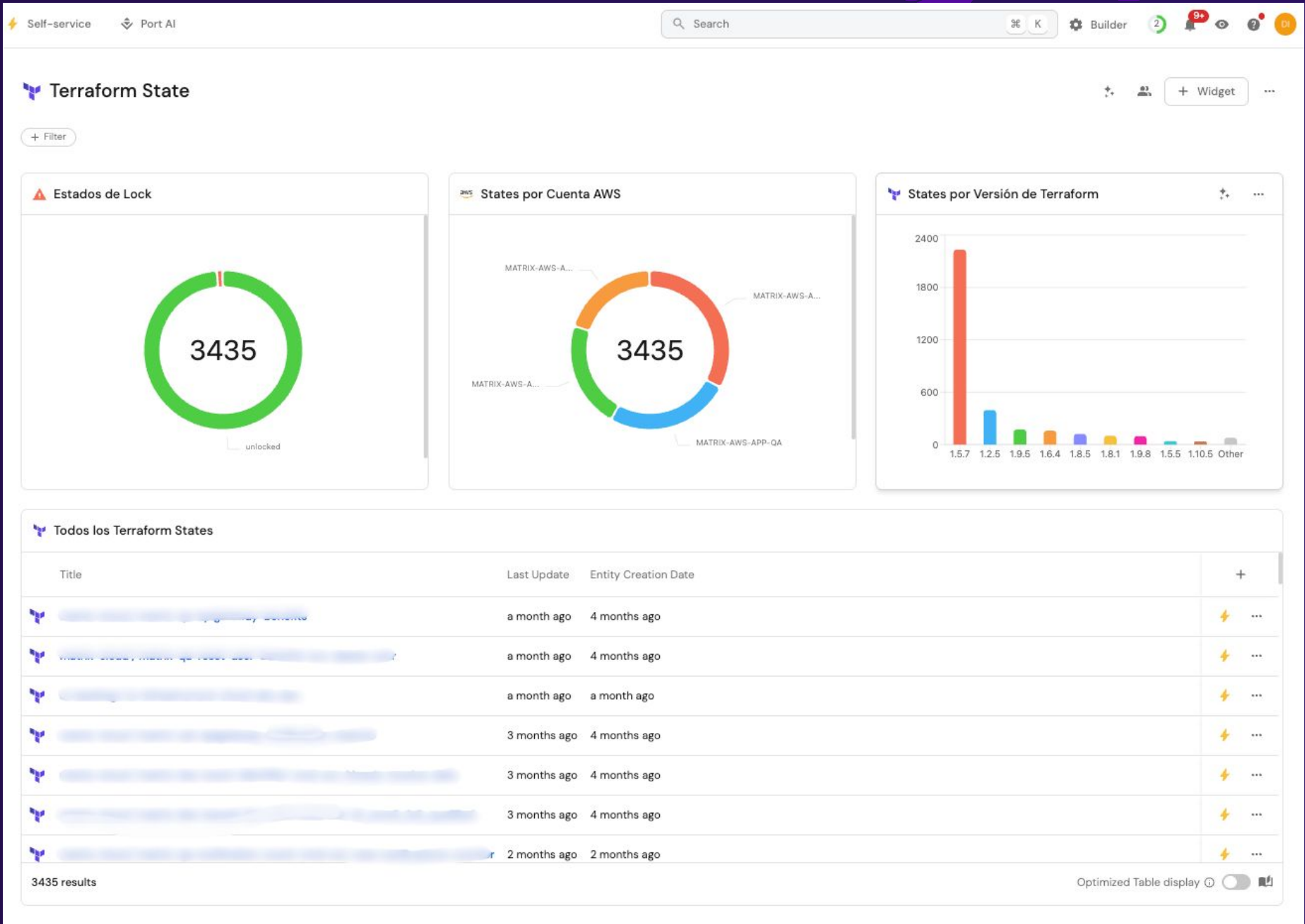
¿Qué equipo es responsable?

¿Cuántos repos en v1 vs v2?



El grafo de relaciones responde en segundos lo que antes tomaba días
Aplicamos principios de observabilidad a IaC para responder preguntas sobre existencia, consumo, versiones y ownership.

- ¿Qué módulos existen?
- ¿Quién los usa y en qué versión?
- ¿Qué equipo es responsable?

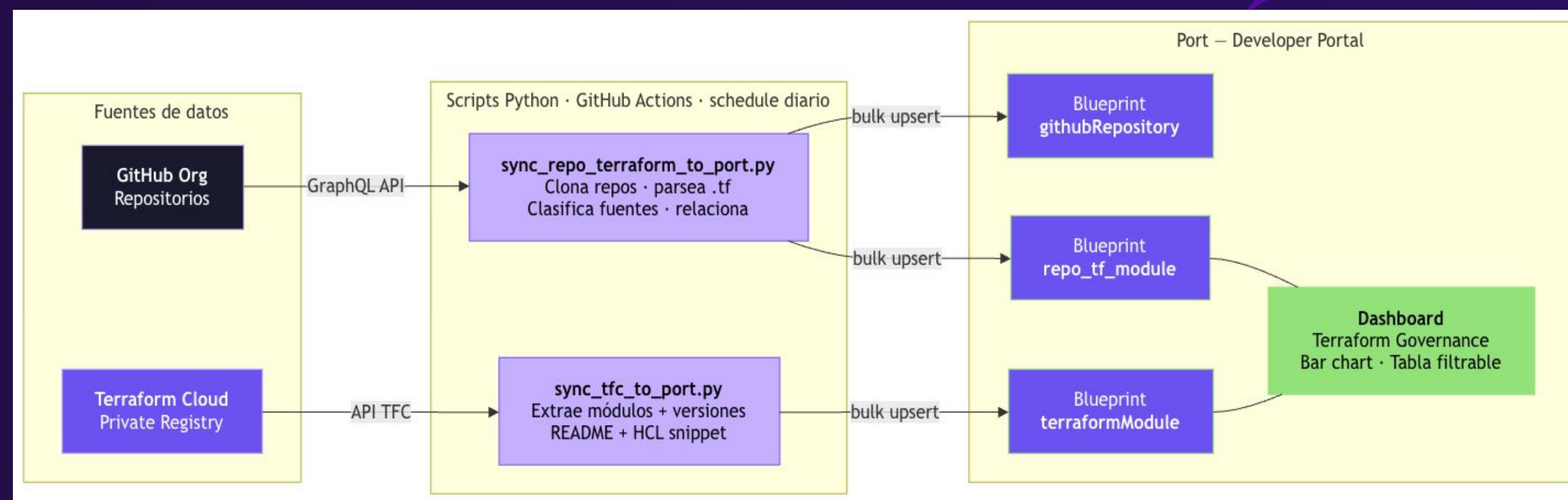


La Arquitectura

Dos fuentes, un modelo común, una vista centralizada

Scripts Python + GitHub Actions scheduleds = catálogo IaC actualizado sin intervención

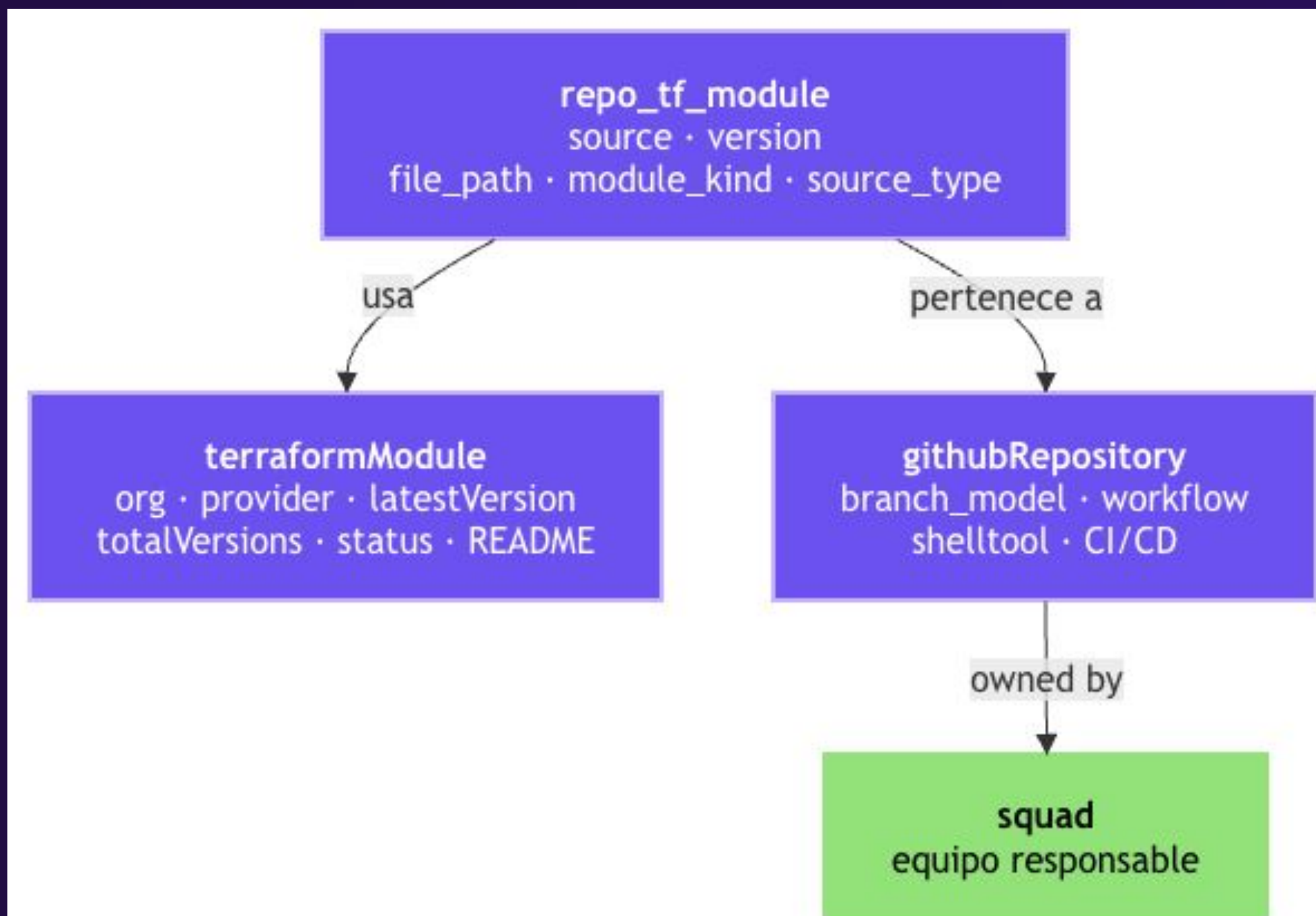
- Blueprints de Port → gestionados con Terraform
- GitHub Actions scheduled workflows → ejecuta ambos scripts
- Port → catálogo unificado de IaC



El Modelo de Datos en Port

Tres blueprints forman el grafo de laC Observability en Port

Cada blueprint es un tipo de entidad con propiedades y relaciones hacia otros blueprints



“Los datos por separado no nos daban governance. El valor apareció cuando empezamos a relacionarlos.”

Sync: TFC Registry → Port

sync_tfc_to_port.py: convierte el registry privado de TFC en entidades Port

Por cada módulo en TFC: extrae metadatos + README + snippet HCL → bulk upsert a Port

- Entrada: TFC_ORG · una o múltiples organizaciones
- Por módulo: metadatos + README + snippet HCL listo para copiar
- Salida: bulk upsert a Port · chunks de 20 · retry automático en 5xx

```
# El corazón del script: por cada módulo en el registry TFC
for module in list_registry_modules(org, TFC_TOKEN):
    entity = {
        "identifier": module["name"],
        "title": module["name"],
        "properties": {
            "organization": org,
            "provider": module["provider"],
            "latestVersion": module["version_statuses"][0]["version"],
            "totalVersions": len(module["version_statuses"]),
            "status": module["status"],
            "usageInstructions": build_usage_snippet(org, module),
            "readme": fetch_module_readme(org, module, TFC_TOKEN),
        },
        "relations": {"definition_repository": find_github_repo(module)}
    }
port_bulk_upsert_entities(PORT_BASE, token, "terraformModule", entities)
```



Sync: GitHub Repos → Port

sync_repo_terraform_to_port.py: detecta qué módulos consume cada repositorio

Shallow clone → scan .tf → parse HCL → clasificar fuente → upsert a Port

- GitHub GraphQL → lista paginada de repos de la org
- Shallow clone → scan recursivo de .tf → parse con python-hcl2
- `classify_module_source()` → detecta si es registry privado ← señala el código
- Bulk upsert de `repo_tf_module` → relaciones a módulo + repo

```
def classify_module_source(source: str) -> tuple[str, str]:
    """Clasifica el source de un módulo: (kind, type)"""
    if source.startswith("./") or source.startswith("../"):
        return "local", "local"
    if source.startswith("app.terraform.io/"):
        return "remote", "private_registry" # ← el que nos importa
    if source.count("/") >= 2:
        return "remote", "registry" # registry público
    return "remote", "unknown"

def terraform_module_idenfifier_from_source(source: str) -> str | None:
    """Extrae el nombre del módulo para relacionarlo en Port"""
    # app.terraform.io/<org>/<name>/<provider> → <name>
    parts = source.split("/")
    if parts[0] == "app.terraform.io" and len(parts) >= 4:
        return parts[2] # ← el nombre del módulo en el registry privado
    return None
```

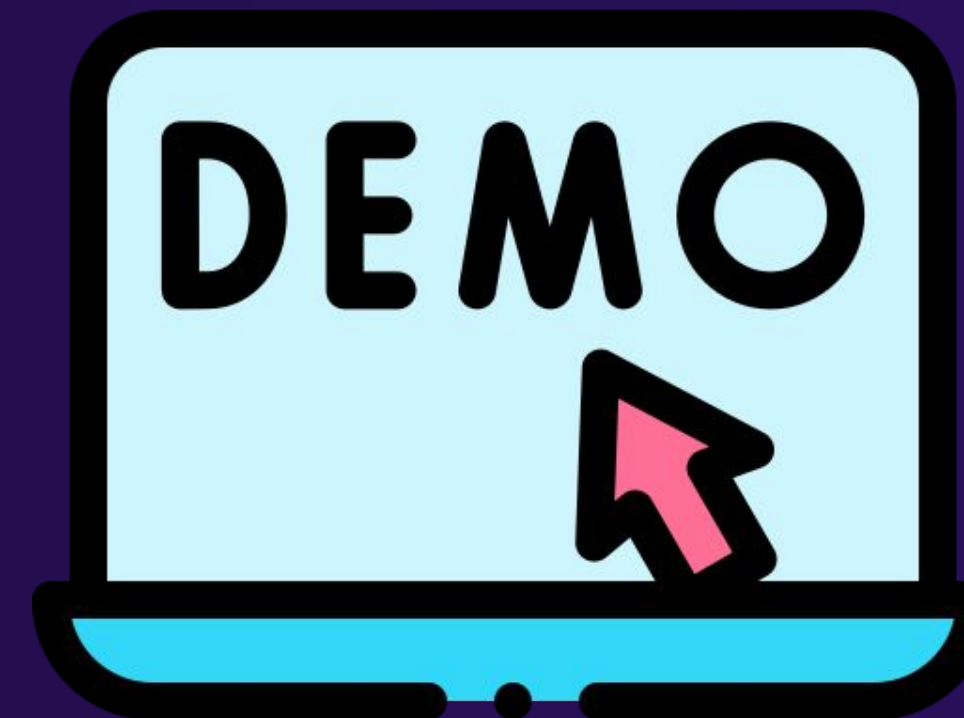


Demo: Terraform Governance Dashboard

¿Puedo retirar v1.2.0 sin romper producción?

Port · Datos reales de producción · Nombres de repos editados por confidencialidad

- Catálogo de módulos TFC → sincronizado automáticamente
- Relaciones: módulo → repos → versiones en uso
- Bar chart de distribución de versiones



¿Qué aprendimos?

Lo más valioso que aprendimos construyendo esto en producción

5 learnings para llevar a tu organización si vas por el mismo camino

1. El grafo es el valor.

Los datos aislados no generan governance.

2. Parsing HCL fue la parte difícil.

Especialmente normalizar y clasificar sources.

3. Gestiona el modelo del IDP como código.

Blueprints de Port con Terraform.

4. Diseña la ingesta para escala.

Bulk upsert + chunking + retries.

5. Automatiza la actualización.

Governance basada en datos actuales, no en inventarios manuales.

El valor no está en saber qué módulos existen. Está en conocer las relaciones: quién los consume, en qué versión y quién es responsable.





Gracias Conectemos

Información de
contacto

Linkedin: /in/dianaalfarobazan



Información de
contacto

Join the conversation: #DODLima #DevOpsDaysLima